

Servomoteurs Série M17 - FICHE TECHNIQUE



Contrôle de mouvement tout-en-un, simple et abordable
De l'éducation à l'innovation



Introduction

Les servomoteurs de la série M17 sont des solutions de contrôle de mouvement tout-en-un qui intègrent un moteur, un pilote de moteur, un contrôleur de mouvement et un encodeur dans un seul boîtier compact. Ces servomoteurs disposent d'une interface de communication RS-485, permettant de chaîner plusieurs unités et de les contrôler à partir d'un seul point de connexion. Disponible en quatre modèles - M17-60, M17-48, M17-40 et M17-34 - la série offre des options flexibles pour répondre à des exigences spécifiques de couple et de taille tout en conservant des caractéristiques de contrôle homogènes. Chaque servomoteur de la série M17 est doté de fonctions de contrôle sophistiquées, notamment plusieurs modes de fonctionnement, des capacités d'auto-étalonnage et une surveillance d'état intégrée via des LED d'indication. Les moteurs peuvent être facilement contrôlés depuis n'importe quelle plateforme, y compris Mac, PC, Raspberry Pi ou Arduino (ne nécessitant qu'un adaptateur RS485 peu coûteux), ce qui les rend idéaux pour les applications éducatives comme industrielles. La série prend en charge un contrôle de position précis avec des profils de mouvement trapézoïdaux, un mode de contrôle en boucle fermée et une gestion complète des erreurs. Avec des dimensions de montage NEMA 17 standardisées, une large plage de tension (12-24V) et un protocole de communication robuste, les servomoteurs de la série M17 constituent une solution fiable et flexible pour les applications exigeant un contrôle de mouvement précis, de la robotique et des machines CNC aux équipements de test automatisés et aux instruments scientifiques.

Envoyez-nous vos commentaires

Si vous trouvez des erreurs dans ce document ou si vous avez des questions, veuillez nous en informer en vous rendant sur : tutorial.gearotons.com/feedback. Nous apprécierions vraiment vos suggestions sur la façon d'améliorer le produit ou la documentation. Merci beaucoup !

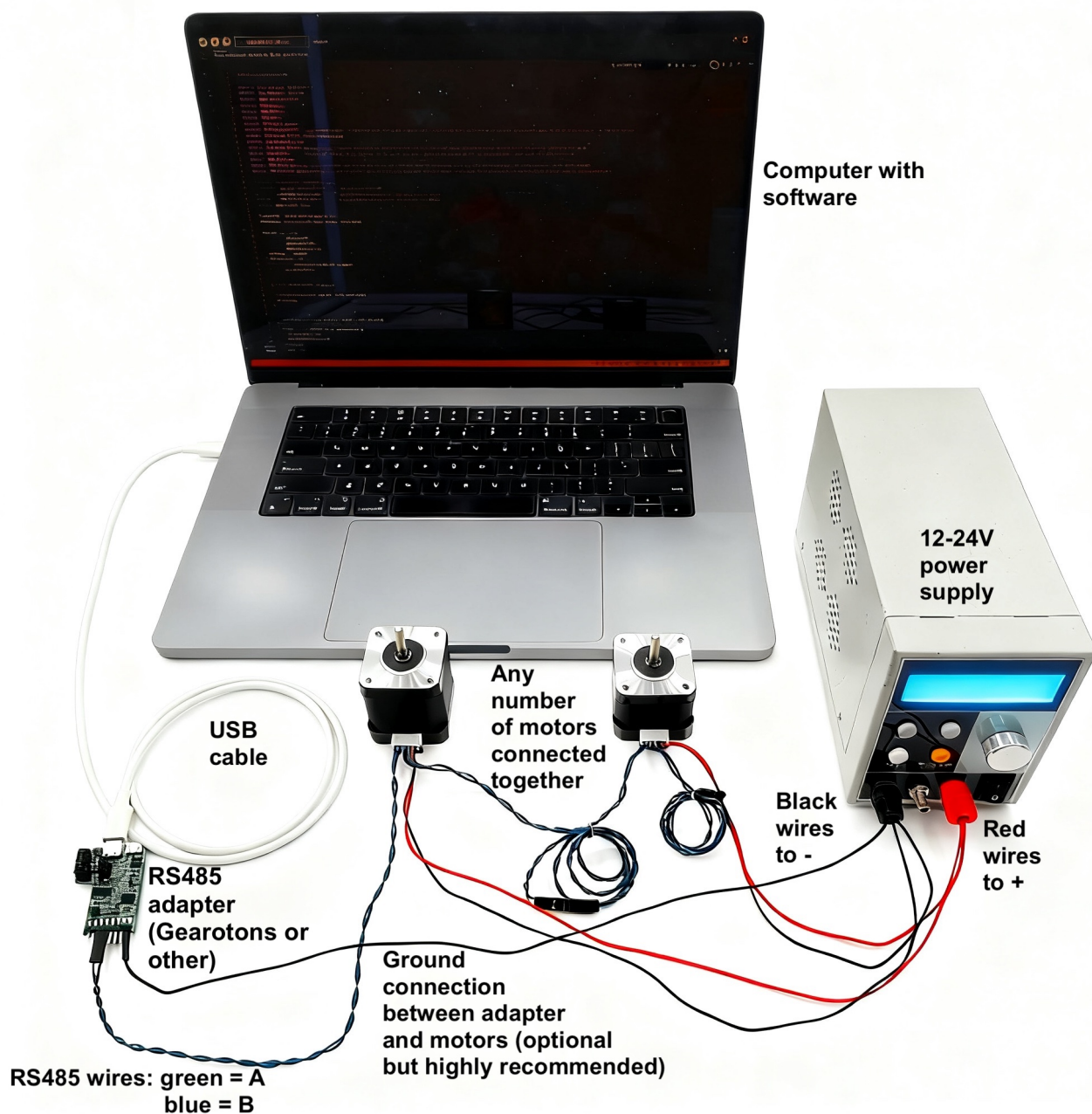


Visitez notre page de commentaires

Caractéristiques principales

- Haut niveau d'intégration combinant un moteur, un pilote de moteur, un système de contrôle de mouvement et un encodeur
- Contrôlez un nombre illimité de moteurs à partir d'un seul contrôleur simple via une interface de communication RS-485
- Contrôle par des commandes de haut niveau telles que "Enable mosfets" (activer les MOSFETs) et "Trapezoid move" (mouvement trapézoïdal), évitant le contrôle DIR/STEP aux contraintes temporelles critiques
- Format compact, de taille presque identique à celle d'un moteur pas à pas NEMA 17 aux mêmes spécifications (sans protubérances)
- Dimensions de montage NEMA 17 standardisées
- Large plage de tension (12-24V) pour des options d'alimentation flexibles
- Contrôle en boucle fermée de haute précision avec encodeur intégré et boucle de régulation PID fonctionnant à 32 kHz
- Bien plus économe en énergie en contrôle en boucle fermée qu'un moteur pas à pas conventionnel
- Protections intégrées contre les surintensités, les surtensions et les surchauffes
- La vitesse maximale peut atteindre 560 RPM
- Rapport couple/poids identique à celui d'un moteur pas à pas équivalent
- Compatible avec une grande variété d'interfaces et de matériels, tels que Raspberry Pi, Arduino, ESP32, Mac et PC
- Nous nous efforçons de fournir une excellente documentation et des tutoriels pour vous rendre opérationnel rapidement
- Nous fournissons une documentation adaptée aux IA au cas où vous souhaiteriez que votre IA préférée fasse tout le travail à votre place
- Convient à la robotique, aux machines CNC, à l'automatisation, aux instruments scientifiques, aux bancs de test, aux imprimantes 3D et à bien d'autres applications
- Disponible en différentes tailles afin que vous puissiez trouver le couple et le prix adaptés à votre application

Schéma de raccordement



Systeme d'unités

Les servomoteurs de la série M17 utilisent certaines unités internes afin d'effectuer efficacement les calculs liés au mouvement (en arithmétique entière). Il incombe au logiciel de contrôle de prendre en charge plusieurs unités de mesure pour les différentes grandeurs. Nos bibliothèques Python et Arduino gèrent automatiquement les conversions d'unités, vous permettant de travailler avec vos unités préférées. Voici les unités prises en charge pour chaque grandeur :

Grandeur	Unités disponibles
Temps	timesteps, seconds, milliseconds, minutes, microseconds
Position	shaft rotations, degrees, radians, encoder counts
Vitesse	rotations per second, rpm, degrees per second, radians per second, counts per second, counts per timestep
Accélération	rotations per second squared, rpm per second, degrees per second squared, radians per second squared, counts per second squared, counts per timestep squared
Courant	internal current units, milliamps, amps
Tension	millivolts, volts
Température	celsius, fahrenheit, kelvin

Guide de démarrage

Pour vous aider à démarrer avec votre servomoteur de la série M17, nous proposons un guide en ligne complet qui couvre tout, de la configuration initiale aux implémentations avancées du protocole. Ce guide comprend :

- Instructions de configuration pas à pas
- Documentation détaillée du protocole de communication
- Exemples de programmation et extraits de code
- Description des codes d'erreur
- Conseils de dépannage et bonnes pratiques



Cliquez ici pour notre guide de démarrage : tutorial.gearotons.com

LED d'indication et boutons

Le servomoteur possède deux LED d'état (verte et rouge). La LED verte clignote lentement pour indiquer un battement de cœur (heartbeat) et rapidement pour indiquer que c'est le bootloader qui s'exécute et non l'application. La LED rouge s'allume brièvement pour signaler une communication sur le bus et indique les codes d'erreur fatale en clignotant un certain nombre de fois.

Le servomoteur possède deux boutons étiquetés "Reset" (réinitialisation) et "Test". Le bouton Reset réinitialise le microcontrôleur interne et tous les états reviennent à leurs valeurs par défaut. Le bouton Test fait tourner le moteur. Appuyez brièvement pour le faire tourner dans un sens, et appuyez pendant plus de 0,3 seconde puis relâchez pour le faire tourner dans l'autre sens. Maintenez le bouton enfoncé pendant au moins 2 secondes puis relâchez pour faire passer le moteur en mode boucle fermée. Maintenez-le enfoncé pendant plus de 15 secondes puis relâchez pour que le moteur effectue un étalonnage sur lui-même. Notez qu'il tournera pendant l'étalonnage et doit pouvoir tourner librement pour que l'étalonnage réussisse ; retirez donc toute charge avant d'effectuer cette opération.

Aperçu du protocole de communication

Les servomoteurs de la série M17 utilisent le RS-485 pour la communication, ce qui permet de chaîner plusieurs moteurs. Chaque moteur possède une adresse unique de 64 bits attribuée en usine afin de pouvoir être adressé individuellement sur un bus RS-485. Chaque moteur peut également se voir attribuer un alias unique d'un octet (0-251) afin d'économiser des octets dans le paquet de communication tout en conservant un contrôle individuel similaire. Il existe aussi un alias de diffusion, qui est 255. Le protocole prend en charge un ensemble complet de commandes pour le contrôle de mouvement, la configuration et la surveillance d'état. La mise à jour du firmware via l'interface RS-485 est prise en charge. La vitesse de communication est fixée à 230400 baud.

Résumé de référence des commandes

Pour la source de référence à jour de toutes les commandes disponibles, vous pouvez consulter ce document.



https://github.com/tomrodinger/servomotor/blob/main/python_programs/servomotor/motor_commands.json

Vous pouvez également exécuter cette commande :

```
pip3 install servomotor # run this just once to install the library and programs
servomotor_command -c
```

Cela affichera les informations contenues dans le fichier motor_commands.json sous une forme plus lisible et fournira des informations d'utilisation pour envoyer des commandes au moteur depuis la ligne de commande.

Contrôle de base

Commande	Description
Disable MOSFETs	Désactive les MOSFETs (notez que les MOSFETs sont désactivés par défaut après la mise sous tension initiale).
Enable MOSFETs	Active les MOSFETs.
Reset time	Remet le temps absolu à zéro (appelez cette commande en premier, avant d'émettre toute commande de mouvement).
Emergency stop	Effectue un arrêt d'urgence (arrête tout mouvement, désactive les MOSFETs et vide la file d'attente).
Zero position	Définit la position actuelle comme position zéro (l'origine).
System reset	Réinitialise le système et entre dans le bootloader. Le moteur se réinitialise immédiatement et entre dans le bootloader. Si aucune commande n'est envoyée dans un court délai, le moteur quitte le bootloader et exécute l'application depuis le début.

Contrôle de mouvement

Commande	Description
Trapezoid move	Se déplace immédiatement vers la position donnée en utilisant la vitesse actuellement définie (la vitesse est définie par une commande distincte).
Go to position	Se déplace vers la position donnée pendant la durée spécifiée. Une accélération et une décélération sont appliquées pour rendre le mouvement fluide.
Homing	Effectue la prise d'origine du moteur (autrement dit, se déplace jusqu'à une butée puis s'arrête immédiatement).
Go to closed loop	Passe en mode de contrôle de position en boucle fermée.
Move with acceleration	Fait tourner le moteur avec l'accélération spécifiée.
Move with velocity	Fait tourner le moteur à la vitesse spécifiée.
Multimove	Compose plusieurs mouvements à exécuter les uns après les autres. Notez que si la file d'attente se vide après l'exécution de tous les mouvements et que le moteur n'est pas à l'arrêt, une erreur fatale est déclenchée.

Configuration

Commande	Description
Set maximum velocity	Définit la vitesse maximale (non utilisée pour le moment).
Set maximum acceleration	Définit l'accélération maximale.
Start calibration	Démarre un étalonnage, qui détermine les valeurs moyennes des capteurs à effet Hall et vérifie leur bon fonctionnement.
Set maximum motor current	Définit le courant moteur maximal et le courant de régénération maximal. Les valeurs prennent effet immédiatement mais ne sont pas enregistrées en mémoire non volatile ; elles reviennent donc aux valeurs par défaut du firmware après une réinitialisation.
Set safety limits	Définit les limites de sécurité (pour empêcher le mouvement de dépasser les bornes définies).
Test mode	Définit ou déclenche un mode de test particulier. Ceci est peu documenté pour le moment. Ne l'utilisez pas sauf si vous êtes un développeur travaillant sur des cas de test. Modes notables : 0 = efface tous les modes de test ; 1-9 = modes de test du moteur ; 10-13 = modes de test des LED (bloquent l'appareil) ; 14-73 = déclenchent les erreurs fatales 0-59 ; 74 = règle le seuil de protection contre les surtensions à 22 V (test de production ; doit se déclencher avec une alimentation de 24 V) ; 75 = règle le seuil de protection contre les surtensions à 26 V (test de production ; ne doit pas se déclencher avec une alimentation de 24 V) ; 76 = relève le seuil de coupure en surtempérature à environ 90 degrés C (test de marge thermique en production ; le seuil de coupure par défaut est d'environ 80 degrés C). Une réinitialisation du système restaure les seuils de surtension et de surtempérature par défaut.
Set PID constants	Définit les constantes PID de la boucle de régulation qui s'efforce de maintenir la trajectoire de mouvement.
Set max allowable position deviation	Définit le nombre de micropas dont la position réelle du moteur (mesurée par les capteurs à effet Hall) est autorisée à s'écarter de la position désirée. Une erreur fatale est déclenchée en cas de dépassement.
CRC32 control	Active ou désactive la vérification CRC32 des commandes.

État et surveillance

Commande	Description
Get current time	Obtient le temps absolu actuel.
Get n queued items	Obtient le nombre d'éléments actuellement dans la file d'attente de mouvements (si ce nombre devient trop grand, ne mettez plus de commandes de mouvement en file d'attente).
Get hall sensor position	Obtient la position mesurée par les capteurs à effet Hall (il devrait s'agir de la position réelle du moteur ; si tout va bien, elle sera à peu près identique à la position désirée).
Get status	Obtient l'état du moteur.
Control hall sensor statistics	Active ou désactive la collecte des statistiques des capteurs à effet Hall et réinitialise les statistiques.
Get hall sensor statistics	Relit les statistiques recueillies à partir des capteurs à effet Hall. Ceci est utile pour vérifier l'état de santé des capteurs à effet Hall et le bruit dans le système.
Get position	Obtient la position désirée actuelle (qui peut différer légèrement de la position réelle mesurée par les capteurs à effet Hall).
Get comprehensive position	Obtient en une seule fois la position désirée du moteur, la position des capteurs à effet Hall et la position de l'encodeur externe.
Get supply voltage	Obtient la tension mesurée de l'alimentation.
Get max PID error	Obtient les valeurs d'erreur minimale et maximale observées dans la boucle de régulation PID depuis la dernière lecture.
Get temperature	Obtient la température mesurée du moteur.
Get debug values	Obtient des valeurs de débogage, notamment les paramètres de contrôle du moteur, les temps du profileur, les données des capteurs à effet Hall et d'autres informations de diagnostic.
Get communication statistics	Obtient et, éventuellement, réinitialise le compteur d'erreurs CRC32.

Gestion de l'appareil

Commande	Description
Time sync	Envoie le temps maître au moteur afin qu'il puisse synchroniser sa propre horloge (à faire 10 fois par seconde).
Get product specs	Obtient la fréquence de mise à jour (l'inverse du pas de temps).
Detect devices	Détecte tous les appareils connectés sur l'interface RS485. Les appareils s'identifient à un instant aléatoire dans un délai d'une seconde. Une collision est possible mais peu probable. Vous pouvez répéter l'opération si vous soupçonnez une collision (par exemple, si des appareils sont connectés mais n'ont pas été découverts en une à deux secondes).
Set device alias	Définit l'alias de l'appareil.
Get product info	Obtient les informations sur le produit.
Firmware upgrade	Met à jour la mémoire flash du servomoteur. Avant d'émettre une commande de mise à jour du firmware, vous devez effectuer certains calculs, comme illustré dans les exemples.
Get product description	Obtient la description du produit.
Get firmware version	Obtient la version du firmware ou la version du bootloader, selon le mode dans lequel se trouve l'appareil. Cette commande renvoie également les bits d'état, dont le bit de poids faible vous indique si l'appareil se trouve actuellement dans le bootloader (=1) ou dans le firmware principal (=0).
Ping	Envoie une charge utile contenant des données quelconques, et l'appareil répond avec les mêmes données.
Vibrate	Fait varier rapidement la tension du moteur et le fait donc vibrer (ou arrête la vibration).
Identify	Identifiez votre moteur en envoyant cette commande. La LED verte du moteur clignote rapidement pendant 3 secondes.

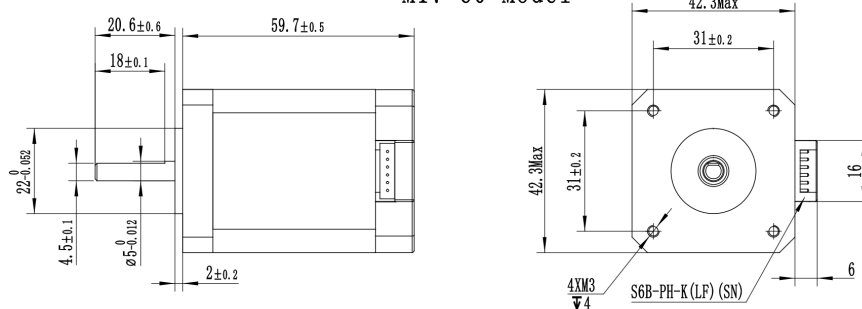
Autres

Commande	Description
Capture hall sensor data	Commence à envoyer les données des capteurs à effet Hall (travail en cours ; n'envoyez pas cette commande).
Read multipurpose buffer	Lit le contenu du tampon polyvalent (ce tampon est utilisé pour les données générées lors de l'étalonnage, lors du passage en mode boucle fermée et lors de la capture des données des capteurs à effet Hall).

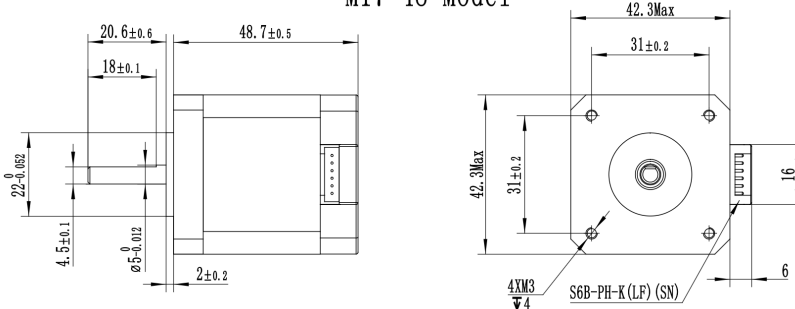
Spécifications mécaniques

Paramètre	M17-60	M17-48	M17-40	M17-34
Dimensions (L x l)	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm	42.2x42.2 mm
Hauteur	59.7 mm	48.7 mm	40.1 mm	33.5 mm
Longueur de l'arbre	20.6 mm	20.6 mm	20.6 mm	20.6 mm
Poids	470g	360g	285g	210g
Classe de protection	IP20	IP20	IP20	IP20

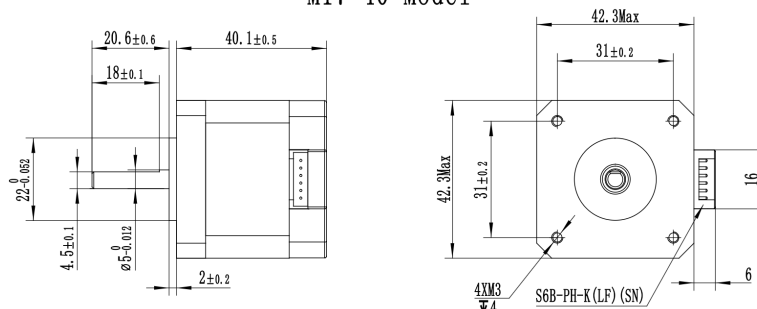
M17-60 Model



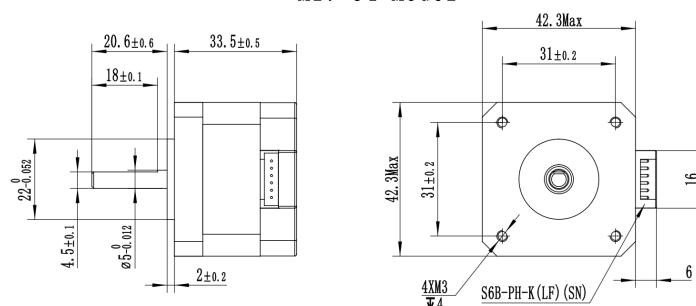
M17-48 Model



M17-40 Model



M17-34 Model



Spécifications techniques

Paramètre	M17-60	M17-48	M17-40	M17-34
Tension de fonctionnement	12-24V	12-24V	12-24V	12-24V
Couple nominal	0.65 N.m	0.55 N.m	0.42 N.m	0.28 N.m
Vitesse maximale	560 RPM	560 RPM	560 RPM	560 RPM
Courant maximal	1.1A	1.1A	1.1A	1.0A
Puissance nominale	26.4W	26.4W	26.4W	24.0W

Conditions de fonctionnement

Paramètre	Spécification
Température de fonctionnement	0C à +80C
Température de stockage	-20C à +60C
Plage d'humidité	20% à 80% RH (sans condensation)
Environnement d'installation	Utilisation en intérieur uniquement

Bibliothèque Python

Notre bibliothèque Python offre une interface complète pour contrôler les servomoteurs de la série M17. La bibliothèque gère tous les protocoles de communication de bas niveau ainsi que les conversions d'unités, facilitant l'intégration du contrôle de servomoteurs dans vos projets Python.

Installation

Installez la bibliothèque servomotor à l'aide de pip :

```
pip3 install servomotor
```

Exemple : contrôle de base du moteur

L'exemple suivant montre comment se connecter à un servomoteur, activer les MOSFETs et effectuer un mouvement trapézoïdal :

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X' # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110" # Serial device path; change if needed (e.g., "COM3" on
# Windows)
DISPLACEMENT_ROTATIONS = 1.0 # 1 rotation
DURATION_SECONDS = 1.0 # 1 second
DELAY_MARGIN = 0.10 # +10% wait margin because the motor's clock is not
# perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
# should prompt you for the serial port or it will use
# the last used port from a file

servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Caractéristiques principales

- Prise en charge de toutes les commandes du moteur
- Conversion automatique des unités pour le temps, la position, la vitesse, l'accélération, le courant, la tension et la température
- Gestion des erreurs et surveillance d'état
- Prise en charge de plusieurs moteurs sur le même bus
- Soyez opérationnel en quelques lignes de code seulement

Documentation

Pour la documentation complète de l'API et d'autres exemples, consultez :
https://github.com/tomrodinger/servomotor/tree/main/python_programs

Bibliothèque Arduino

La bibliothèque Arduino pour les servomoteurs de la série M17 offre une interface facile à utiliser pour contrôler les moteurs depuis des cartes Arduino. La bibliothèque prend en charge la communication série matérielle et inclut toutes les fonctions essentielles de contrôle du moteur.

Installation

Installez la bibliothèque via le gestionnaire de bibliothèques de l'IDE Arduino :

- 1. Ouvrez l'IDE Arduino
- 2. Allez dans Croquis → Inclure une bibliothèque → Gérer les bibliothèques
- 3. Recherchez "Servomotor"
- 4. Trouvez la bibliothèque nommée "Servomotor by Gearotons" et cliquez sur Installer



Installation manuelle

Autrement, si vous êtes un expert, vous pouvez récupérer le code de la bibliothèque Arduino ici : https://github.com/tomrodinger/Servomotor_Arduino_Library

Exemple : contrôle de base du moteur

Cet exemple montre comment initialiser un moteur, activer les MOSFETs et effectuer un mouvement trapézoïdal :

```

// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
// enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X' // Device alias
#define BAUD 230400 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f // 1 second
#define TOLERANCE_PERCENT 10 // +10% wait margin because the motor's clock is not
// perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4 // TX pin to RS485 transceiver
#define RS485_RXD 5 // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}

```

Notre mission

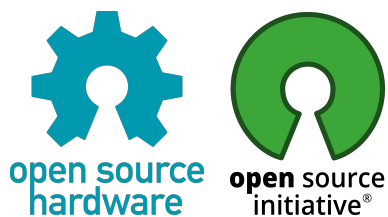
Chez Gearotons, nous croyons que l'avenir appartient à ceux qui comprennent l'intelligence artificielle et les systèmes d'automatisation et savent travailler à leurs côtés. Notre mission est de donner les moyens d'agir à la prochaine génération d'innovateurs en rendant la technologie avancée de contrôle de mouvement accessible, compréhensible et abordable pour les makers, les enseignants et les ingénieurs du monde entier. Nous nous engageons à faire tomber les barrières qui ont traditionnellement séparé les étudiants et les amateurs des outils d'automatisation de qualité professionnelle. En proposant des solutions intégrées qui éliminent la complexité sans sacrifier les capacités, nous rendons possibles des expériences d'apprentissage pratiques qui préparent les jeunes esprits à un monde où la créativité humaine et la précision des machines travaillent en harmonie. Notre conviction fondamentale est que chaque étudiant devrait avoir la possibilité d'expérimenter avec les briques de base de l'automatisation moderne - de la robotique et des systèmes CNC aux instruments scientifiques. Grâce à nos partenariats éducatifs et à notre approche open source, nous formons une génération qui ne se contente pas d'utiliser la technologie, mais qui la comprend et la façonne véritablement. Ensemble, nous bâtissons les fondations pour les innovateurs qui définiront un avenir extraordinaire.

Open Source

Nous croyons en un monde meilleur grâce à la technologie. Les logiciels, le firmware et les fichiers de conception des PCB sont disponibles ici :



<https://github.com/tomrodinger/servomotor>



Version de la fiche technique : 1.9 Date de publication : Jun. 29, 2026

(c) 2026 Toutes les spécifications peuvent être modifiées sans préavis.